

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №1
по курсу «Логика и ОА в ИЗ»
на тему «Простые структуры данных»»

Выполнили:
студенты группы 25ВВВ1
Новиков И.А.
Евстратов В.С.

Принял:
к.т.н., доцент Юрова О.В.

Пенза 2026

Цель работы

Изучение основных статических и динамических структур данных языка Си (статические и динамические массивы, двумерные массивы, структуры), освоение методов работы с динамической памятью (malloc, free), генерации случайных чисел (rand, srand) и файлового ввода/вывода для сохранения и загрузки структурированных данных.

Лабораторное задание

Разработать программу вычисления функции

Задание 1: Вычисление разности между максимальным и минимальным элементами одномерного статического массива.

Задание 2: Инициализация одномерного массива случайными числами в заданном диапазоне.

Задание 3: Создание одномерного динамического массива, размер которого вводится с клавиатуры.

Задание 4: Вычисление суммы элементов по столбцам в двумерном динамическом массиве (матрице).

Задание 5: Органайзер базы данных студентов на основе структур student с возможностью добавления, просмотра, поиска по различным полям (фамилия, имя, факультет, номер зачетной книжки) и сохранением/загрузкой данных из файла students.dat.

Описание метода решение задач

Задание 1 (task1): Массив из 10 элементов заполняется случайными числами в диапазоне [-50, 50] с использованием формулы `rand() % 101 - 50`. Поиск минимума и максимума производится одним проходом по массиву, после чего рассчитывается их разность (`max - min`).

Задание 2 (task2): Демонстрация базовой инициализации генератора псевдослучайных чисел `srand(time(NULL))` и заполнения массива целыми числами в диапазоне [0, 99] через операцию получения остатка от деления `rand() % 100`.

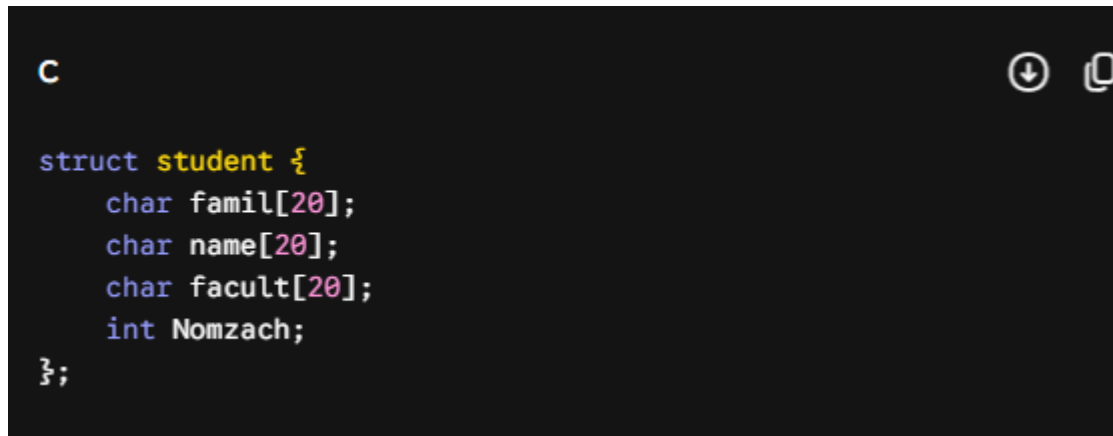
Задание 3 (task3): Осуществляется динамическое выделение памяти под `$n$` элементов целого типа с помощью функции `malloc(n * sizeof(int))`. После заполнения массива случайными числами и его вывода выделенная память обязательно высвобождается с помощью функции `free()`.

Задание 4 (task4):

Создается двумерный динамический массив размерностью `$rows \times cols$` как массив указателей на строки. Память выделяется последовательно для

строк и элементов каждой строки. Вычисляются суммы по столбцам путем суммирования элементов $a[i][j]$ по индексам строк i при фиксированном индексе столбца j . По окончании работы высвобождается память всех строк и массива указателей.

Задание 5 (task5): Описывается пользовательская структура



```
C

struct student {
    char famil[20];
    char name[20];
    char facult[20];
    int Nomzach;
};
```

ЛИСТИНГ

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_STUDENTS 100
#define DB_FILE "students.dat"

struct student {
    char famil[20];
    char name[20];
    char facult[20];
    int Nomzach;
};

int load_students(struct student *stud) {
    FILE *f;
    int count = 0;
    f = fopen(DB_FILE, "r");
    if (f == NULL) return 0;
    while (count < MAX_STUDENTS &&
           fscanf(f, "%19s %19s %19s %d",
```

```

                                stud[count].famil,
stud[count].name,
                                stud[count].facult,
&stud[count].Nomzach) == 4) {
    count++;
}
fclose(f);
return count;
}

```

```

void save_students(struct student *stud, int count) {
    FILE *f;
    int i;
    f = fopen(DB_FILE, "w");
    if (f == NULL) {
        printf("Error saving database\n");
        return;
    }
    for (i = 0; i < count; i++) {
        fprintf(f, "%s %s %s %d\n",
                stud[i].famil, stud[i].name,
                stud[i].facult, stud[i].Nomzach);
    }
    fclose(f);
    printf("Saved %d students to %s\n", count,
DB_FILE);
}

```

```

void task1(void) {
    int a[10];
    int i, min, max;

    srand(time(NULL));
    printf("Array: ");
    for (i = 0; i < 10; i++) {
        a[i] = rand() % 101 - 50;
        printf("%4d", a[i]);
    }

    min = a[0];

```

```

    max = a[0];
    for (i = 1; i < 10; i++) {
        if (a[i] < min) min = a[i];
        if (a[i] > max) max = a[i];
    }

    printf("\nMax: %d\n", max);
    printf("Min: %d\n", min);
    printf("Difference: %d\n", max - min);
}

void task2(void) {
    int a[10];
    int i;

    srand(time(NULL));
    printf("Random array [0, 99]:\n");
    for (i = 0; i < 10; i++) {
        a[i] = rand() % 100;
        printf("%4d", a[i]);
    }
    printf("\n");
}

void task3(void) {
    int n, i;
    int *a;

    printf("Enter array size: ");
    scanf("%d", &n);

    a = (int *)malloc(n * sizeof(int));
    if (a == NULL) {
        printf("Memory allocation error\n");
        return;
    }

    srand(time(NULL));
    printf("Array of %d elements:\n", n);
    for (i = 0; i < n; i++) {

```

```

        a[i] = rand() % 101 - 50;
        printf("%4d", a[i]);
    }
    printf("\n");

    free(a);
}

void task4(void) {
    int rows, cols, i, j, sum;
    int **a;

    printf("Enter number of rows: ");
    scanf("%d", &rows);
    printf("Enter number of columns: ");
    scanf("%d", &cols);

    a = (int **)malloc(rows * sizeof(int *));
    for (i = 0; i < rows; i++)
        a[i] = (int *)malloc(cols * sizeof(int));

    srand(time(NULL));
    printf("\nMatrix %dx%d:\n", rows, cols);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            a[i][j] = rand() % 41 - 20;
            printf("%4d", a[i][j]);
        }
        printf("\n");
    }

    printf("\nColumn sums:\n");
    for (j = 0; j < cols; j++) {
        sum = 0;
        for (i = 0; i < rows; i++)
            sum += a[i][j];
        printf("Column %d: %d\n", j + 1, sum);
    }

    for (i = 0; i < rows; i++)

```

```

        free(a[i]);
    free(a);
}

void task5(void) {
    struct student stud[MAX_STUDENTS];
    char search[20];
    int count, i, found, field, cmd;
    int search_num;
    char *val;

    count = load_students(stud);
    printf("Loaded %d students from database\n\n",
count);

    while (1) {
        printf("\n1 - Add student\n");
        printf("2 - List all\n");
        printf("3 - Search\n");
        printf("4 - Save and exit\n");
        printf("Choice: ");
        scanf("%d", &cmd);

        if (cmd == 1) {
            if (count >= MAX_STUDENTS) {
                printf("Database is full\n");
                continue;
            }
            printf("  Last name: ");
            scanf("%19s", stud[count].famil);
            printf("  First name: ");
            scanf("%19s", stud[count].name);
            printf("  Faculty: ");
            scanf("%19s", stud[count].facult);
            printf("  Record book number: ");
            scanf("%d", &stud[count].Nomzach);
            count++;
            printf("Added. Total: %d\n", count);

            } else if (cmd == 2) {

```

```

        if (count == 0) {
            printf("Database is empty\n");
            continue;
        }
        for (i = 0; i < count; i++) {
            printf("%d) %s %s, faculty %s, record
book %d\n",
                                i      +      1,      stud[i].famil,
stud[i].name,
                                stud[i].facult,
stud[i].Nomzach);
        }

    } else if (cmd == 3) {
        printf("Search by:\n");
        printf("1 - Last name\n");
        printf("2 - First name\n");
        printf("3 - Faculty\n");
        printf("4 - Record book number\n");
        printf("Choice: ");
        scanf("%d", &field);

        found = 0;
        if (field >= 1 && field <= 3) {
            printf("Enter search value: ");
            scanf("%19s", search);
            for (i = 0; i < count; i++) {
                val = NULL;
                if      (field == 1)      val =
stud[i].famil;

                else if (field == 2)      val =
stud[i].name;

                else val = stud[i].facult;
                if (strcmp(val, search) == 0) {
                    printf("Found: %s %s, faculty
%s, record book %d\n",
                                stud[i].famil,
stud[i].name,
                                stud[i].facult,
stud[i].Nomzach);

                    found = 1;
                }
            }
        }
    }
}

```

```

        }
    }
    } else if (field == 4) {
        printf("Enter record book number: ");
        scanf("%d", &search_num);
        for (i = 0; i < count; i++) {
            if (stud[i].Nomzach ==
search_num) {
                printf("Found: %s %s, faculty
%s, record book %d\n",
stud[i].name,
stud[i].Nomzach);
                found = 1;
            }
        }
    }
    if (!found)
        printf("Student not found\n");

    } else if (cmd == 4) {
        save_students(stud, count);
        break;
    }
}
}

```

```

int main(void) {
    int choice;

    printf("Lab 1 - Simple data structures\n");
    printf("1 - Max-min difference\n");
    printf("2 - Random array\n");
    printf("3 - Dynamic array\n");
    printf("4 - Matrix column sums\n");
    printf("5 - Student search\n");
    printf("Choice: ");
    scanf("%d", &choice);
}

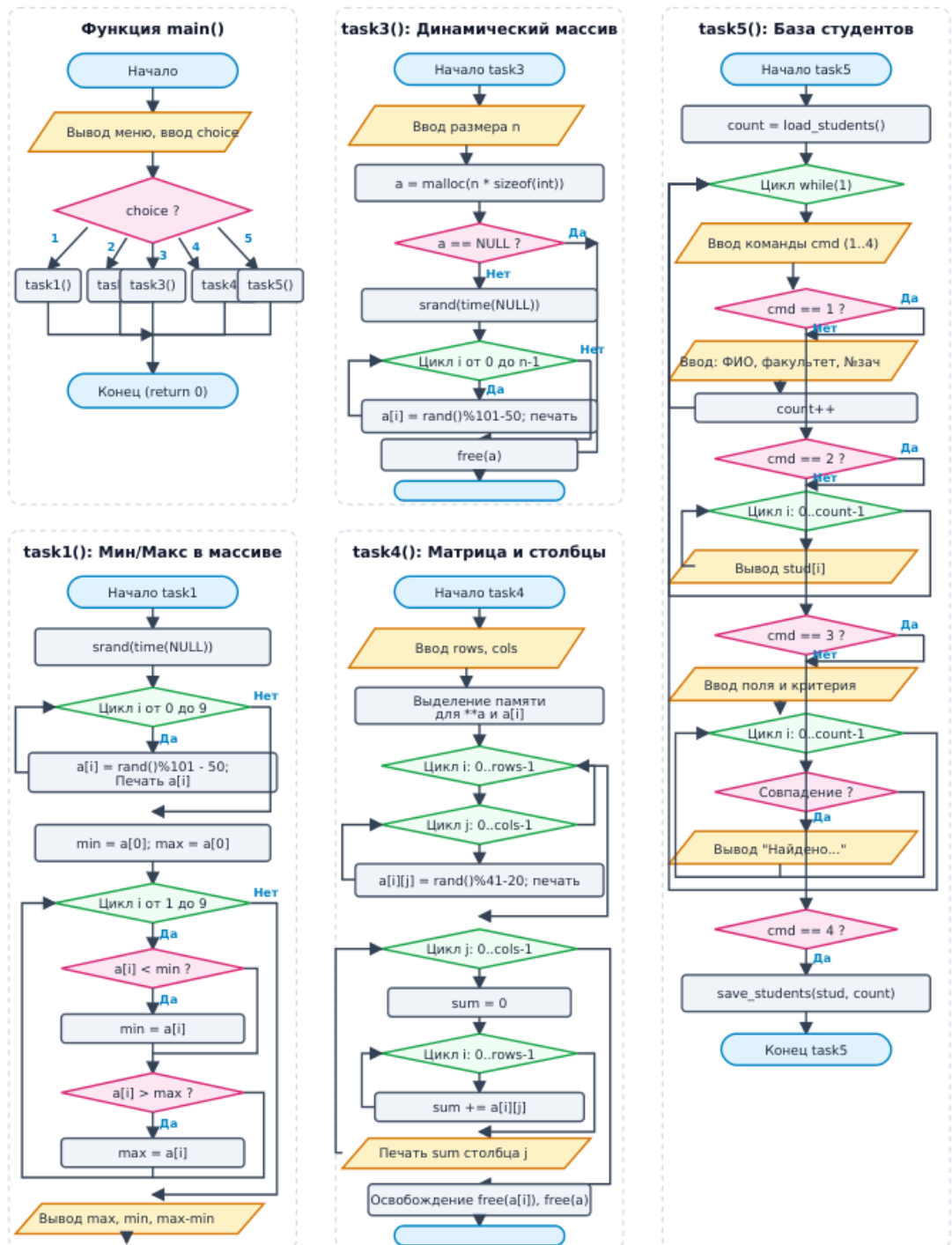
```

```
switch (choice) {
    case 1: task1(); break;
    case 2: task2(); break;
    case 3: task3(); break;
    case 4: task4(); break;
    case 5: task5(); break;
    default: printf("Invalid choice\n"); break;
}

return 0;
```

Блок-схема

Блок-схема программы к Лабораторной работе №1 «Простые структуры данных» (Главная функция main и ключевые задачи)



Пояснительный текст к программе

Генерация случайных чисел: Использование функций `srand(time(NULL))` и `rand()` гарантирует получение разных случайных последовательностей при каждом новом запуске программы.

Динамическое управление памятью: Для работы с одномерными и двумерными массивами неопределенного заранее размера используется функция `malloc()`. По завершении вычислений вся выделенная память корректно освобождается с помощью `free()`, что исключает утечки памяти.

Структуры данных: Структура `struct student` позволяет объединить разнородные типы данных (строки символов и целые числа) в единый объект, удобный для обработки.

Файловый ввод/вывод: Организован механизм сохранения данных в текстовый файл `students.dat` и их считывание при старте функции, что обеспечивает энергонезависимость базы данных.

Результаты работы программы



```
Консоль отладки Microsoft Visual Studio
Lab 1 - Simple data structures
1 - Max-min difference
2 - Random array
3 - Dynamic array
4 - Matrix column sums
5 - Student search
Choice: 1
Array: -33 44 22 43 -12 41 -20 -14 20 11
Max: 44
Min: -33
Difference: 77
C:\Users\Xeon\Downloads\progr\laba3-4\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (процесс 17292) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio

Lab 1 - Simple data structures
1 - Max-min difference
2 - Random array
3 - Dynamic array
4 - Matrix column sums
5 - Student search
Choice: 4
Enter number of rows: 3
Enter number of columns: 3

Matrix 3x3:
-6 13 4
17 -8 14
5 6 12

Column sums:
Column 1: 16
Column 2: 11
Column 3: 30

C:\Users\Xeon\Downloads\progr\laba3-4\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (процесс 18164) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно: _
```

C:\Users\Xeon\Downloads\progr\laba3-4\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe

```
1 - Add student
2 - List all
3 - Search
4 - Save and exit
Choice: 1
Last name: Ivanov
First name: Ivan
Faculty: FIT
Record book number: 250101
Added. Total: 1

1 - Add student
2 - List all
3 - Search
4 - Save and exit
Choice: 3
Search by:
1 - Last name
2 - First name
3 - Faculty
4 - Record book number
Choice: 1
Enter search value: Ivanov
Found: Ivanov Ivan, faculty FIT, record book 250101

1 - Add student
2 - List all
3 - Search
4 - Save and exit
Choice:
```

Выводы

В ходе выполнения лабораторной работы №1 были изучены:

1. Способы работы со статическими и динамическими массивами в языке Си.
2. Функции динамического выделения и освобождения памяти (malloc, free).
3. Использование генератора псевдослучайных чисел rand() и инициализации srand(time(NULL)) для формирования входных данных.
4. Способы объявления и использования структур (struct) для группировки данных, а также алгоритмы поиска по полям структур.
5. Механизмы сохранения и загрузки данных через текстовые файлы.